포트폴리오 2026 / 개발자

# 직접 만든 AI 작업 환경으로 서비스 하나를 혼자 기획부터 운영까지 굴렸습니다.

여러 AI 에이전트가 원본 · 규칙 · 기록을 함께 쓰는 개인 작업 환경(JH Personal OS)을 직접 만들어 매일 씁니다.
그 환경으로 온라인 스토어를 혼자 열어 기획 · 개발 · 광고 · 운영을 맡았고 판매 6일째 하루 1,378건을 처리했습니다.
기존 시스템을 고칠 수 없는 조건에서 공공기관 RAG 챗봇과 사내 문서 MCP 검색 서버도 만들었습니다.



**숫자로 보면**

**1,378건**
하루 최대 주문 · 혼자 운영

**0 → 13건**
남원 청년 일자리 검색 결과 · 지역 필터 개선

**1,809건**
복지사업 RAG 검색 데이터 · 매일 갱신

**수상 3회**
은상 · 캡스톤 우수상 · 기관 표창

**수상**

2025 컴퓨터인공지능학부 작품경진대회 은상
2025 캡스톤디자인 우수상
2025 전북특별자치도사회서비스원 성과보고회 표창


## 배지혁

전북대학교 컴퓨터공학부 / qowlgur121@gmail.com / github.com/qowlgur121 / qowlgur121.github.io

# 소개

학력 · 현장실습 · 기술

### 학력

| | |
|---|---|
| **전북대학교** | 컴퓨터공학부 공학사 · 2022.03 ~ 2026.02 |
| **성적** | 전체 평점 3.71/4.5 · 전공 평점 3.83/4.5<br>3학년 2학기 전 과목 A+ · 학과 1등 (4.50/4.5)<br>4학년 1학기 4.41 · 2학기 4.17 |

### 현장실습

| | |
|---|---|
| **(주)텔로스** | 백엔드 개발 현장실습 · 2025.07 ~ 2025.08 |

### 수상

| | |
|---|---|
| **2025.12** | 작품경진대회 은상 · 전북대 컴퓨터인공지능학부 |
| **2025.12** | 캡스톤디자인 우수상 · 전북대학교 |
| **2025.11** | 성과보고회 표창 · 전북특별자치도사회서비스원 |

### 자격증 · 활동

| | |
|---|---|
| **자격증** | 정보처리기사 · SQLD · ADsP |
| **JAVICE** | SW 창업동아리 · 2023.09 ~ 2025.12 |
| **ALPS** | 알고리즘 동아리 스터디 멘토 · 2022.03 ~ 2023.07 |

### 기술

| | |
|---|---|
| **Java** | 현장실습 · OpenContext · JeonjuEats 백엔드 |
| **Spring Boot** | JPA · Spring Security · JWT 포함 |
| **Python** | 주문 자동화 시스템 · 운영 스크립트 |
| **FastAPI** | 복지 AI 챗봇 백엔드 · SSE 스트리밍 |
| **PostgreSQL** | pgvector 벡터 검색 · OpenContext 메타데이터 |
| **Docker** | Docker Compose · Nginx로 서버 배포 |

### AI와 일하는 방식

| | |
|---|---|
| **설계 · 검증은 직접** | AI 코딩 도구(Claude Code · Codex)로 구현 속도를 높이고, 무엇을 만들지와 결과가 맞는지는 직접 설계하고 테스트로 확인합니다. |
| **계획 → 승인** | 바로 실행시키지 않고 계획과 위험을 먼저 받은 뒤 승인한 것만 실행합니다. |
| **만들기 ≠ 검토** | 만드는 세션과 검토 세션(ChatGPT 웹)을 나누고, 검토 쪽은 문제만 찾고 파일을 고치지 않게 합니다. |
| **자유도 줄이기** | 결제 · 발주 · 개인 기록처럼 틀리면 손해가 나는 곳은 AI 판단 대신 규칙과 허용된 명령으로만 처리합니다. |
| **검사로 강제** | AI가 놓치기 쉬운 지점은 문서 규칙이 아니라 자동 검사로 고정합니다 (문제 해결 6). |

01

# 주문 자동화 시스템과 온라인 스토어 운영

대표 프로젝트 · 개요와 구조

개인 · 기획부터 운영까지 혼자

2026.09 ~ 2026.10

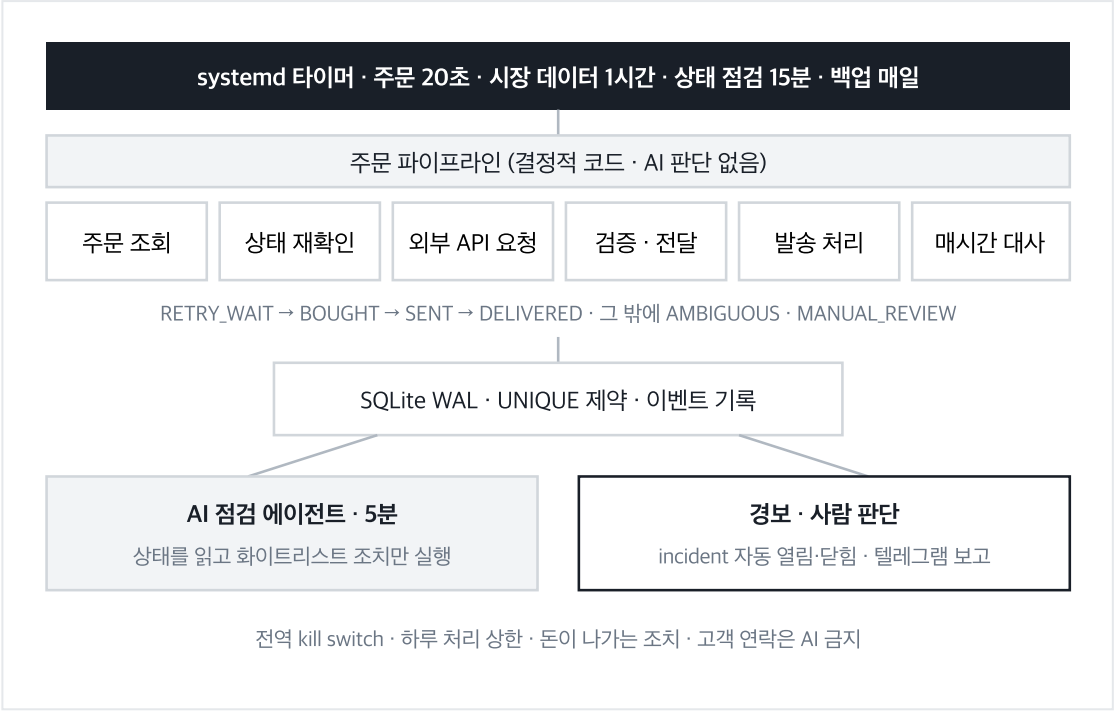
**온라인 판매를 직접 기획하고, 주문이 늘어나도 혼자 처리할 수 있는 서버를 먼저 만들었습니다. 결제 확인부터 발주·전달·정산 대조를 자동화해 판매 6일째 하루 1,378건을 처리했습니다.**

| | |
|---|---|
| **Stack** | Python · 외부 의존성 최소화 · SQLite WAL · unittest |
| **Infra** | Oracle Cloud ARM VM · Ubuntu · systemd 타이머 · 전용 계정과 ProtectSystem=strict |
| **연동** | 네이버 커머스·검색광고 API · 외부 주문 API · Telegram Bot |
| **AI** | AI 점검 에이전트 (5분) · 운영 조회 에이전트 (읽기 전용) |
| **확장** | 단일 서버 기준 설계. 부하 근거가 생기면 PostgreSQL로 옮기는 기준을 문서로 정해 둠 |

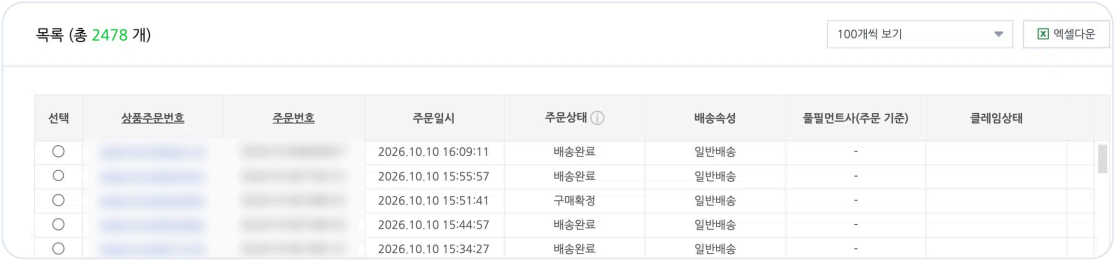
**1,378건**
하루 최대 주문 (10/07)

**2,467건**
9일간 누적 주문

**약 40%**
마진 대비 주문당 광고비 (10/07)



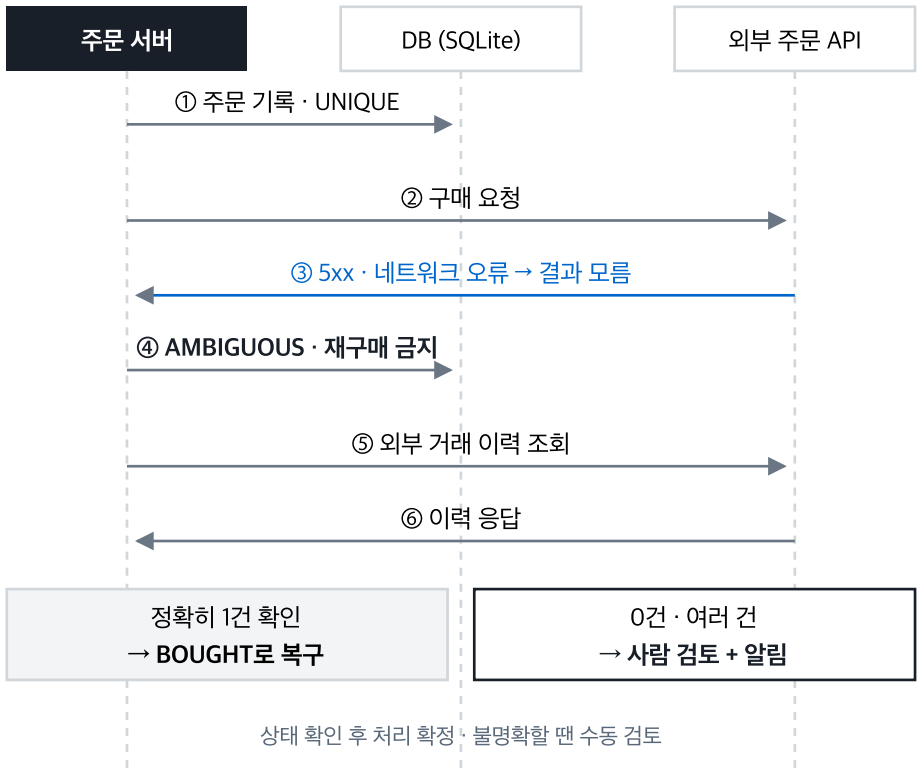

시스템 구조. 회색 상자는 AI 에이전트 · 테두리 상자는 사람 판단




판매자센터 주문 목록 (주문번호는 가림)

# 외부 API가 응답하지 않아도 중복 주문을 막는 상태 관리

문제 해결 1




구매 결과를 모를 때의 처리 순서

### 문제

외부 주문 API의 응답이 끊기면 요청의 처리 여부를 알 수 없었습니다. 재시도만으로 복구하면 하나의 주문이 중복 처리될 수 있어, 확인 가능한 상태 관리가 필요했습니다.

### 설계 판단

| | |
|---|---|
| **자동 재시도** | 동일 요청이 중복 실행될 수 있음 |
| **즉시 실패** | 실제 처리 여부와 내부 상태가 어긋날 수 있음 |
| **선택한 방식** | **상태 보류 → 거래 이력 확인 → 결과 확정** |

처리 결과가 불명확하면 **AMBIGUOUS** 상태로 보류했습니다. 외부 거래 이력에서 일치하는 처리 내역을 찾은 뒤 복구하고, 판단이 어려운 건은 수동 확인으로 넘겼습니다.

주문번호 **UNIQUE** 제약과 조건부 UPDATE로 동일 주문의 상태 전이를 보호했습니다.

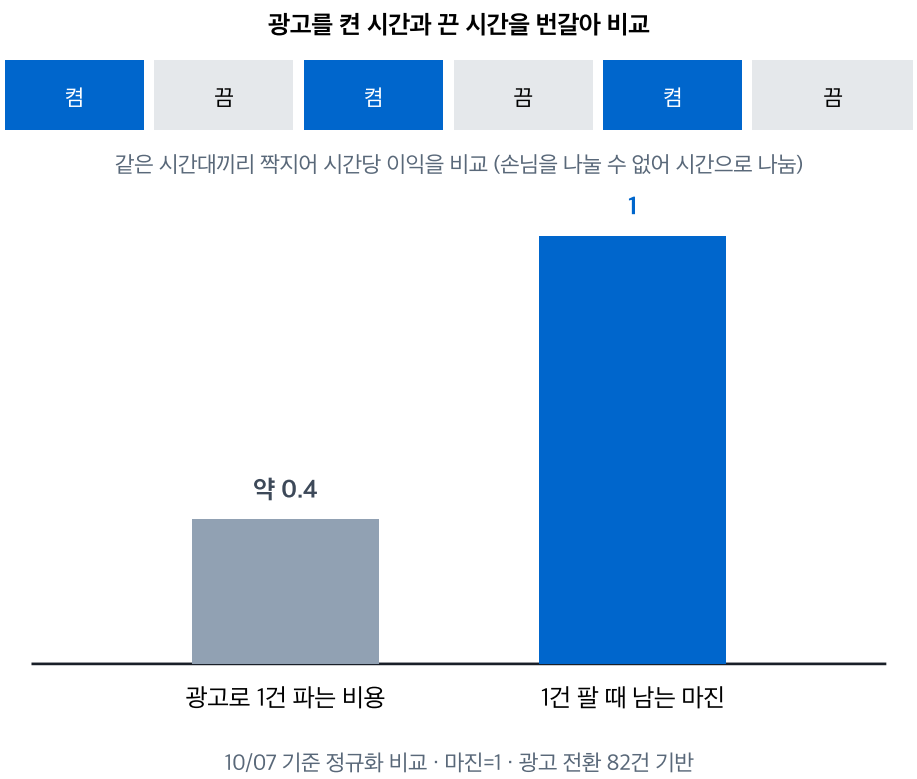
### 결과

불명확한 주문을 무조건 재실행하지 않고 **대조 후 복구하는 경로**를 마련했습니다. 판매 6일째 하루 최대 1,378건을 자동 처리했고, 중복 처리 관련 예외는 회귀 테스트에 반영했습니다.

운영 환경 · Oracle Cloud ARM 1 OCPU · 6GB 1대 · SQLite WAL · 20초 주기 조회 · 하루 최대 1,378건(시간당 평균 약 57건)
핵심 로직 예제 코드 · github.com/qowlgur121/order-fulfillment-guard

# 광고를 켜야 할 시점을 주문당 이익으로 판단했습니다

문제 해결 2




광고 판단 방식과 하루 기준 비교

**문제**

광고 클릭 수나 노출량이 늘어도 실제로 이익이 남는지는 별개의 문제였습니다. 예산을 늘리기 전에 **주문당 비용과 마진**을 함께 확인할 기준이 필요했습니다.

**해결**

광고를 켠 시간과 끈 시간을 번갈아 기록하고, 비슷한 시간대의 **주문당 광고비·마진·시간당 이익**을 비교했습니다.

손익 기준에 미달하면 광고를 끄도록 규칙을 두고, 주문 유입경로와 판매 흐름을 함께 수집해 예산 판단에 반영했습니다.

> **핵심 선택** · 클릭 수 최적화보다 실제 주문 수익성을 우선

**확인한 결과**

**10월 7일 광고 전환 주문 82건**에서 주문당 광고비가 마진의 약 40% 수준임을 확인했습니다. 이를 예산 판단 근거로 활용했습니다.

조건 · 10월 7일 광고 전환 주문 기준 · 판매자센터/광고 API 집계
같은 날 전체 주문 1,378건과 광고 전환 주문은 별도 지표

02

# 공공 복지 AI 챗봇 고도화 · RAG

대표 프로젝트 · 개요와 구조

산학 프로젝트 · 3인 팀 팀장

2025.09.22 ~ 2025.12.05

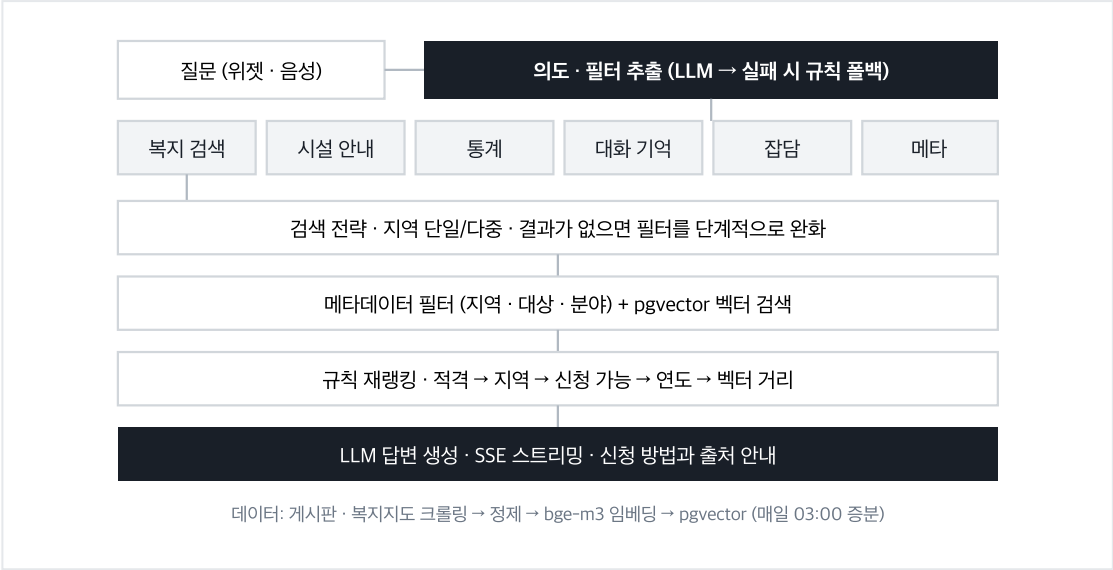
**표현이 조금만 달라도 답을 못 찾던 기관 챗봇을 의도 판단과 RAG 검색으로 다시 만들었습니다. 팀장으로 기관과 요구사항을 맞추고 개발을 맡았습니다.**

| | |
|---|---|
| **협력** | 전북특별자치도사회서비스원 |
| **Backend** | FastAPI 0.115 · SQLAlchemy 2 · APScheduler · SSE · Docker Compose · Nginx |
| **AI · 검색** | LangChain 0.3 · bge-m3 임베딩 · PostgreSQL + pgvector · LLM 클라우드/로컬 선택 |
| **Frontend** | 관리자 React 19 · Vite · Tailwind · Recharts / 위젯 Vanilla JS Custom Element |
| **역할** | **내 담당** 요구사항 협의 · 설계 · 백엔드 · 검색 · 관리자 · 위젯 · 발표<br>**팀원** 자료 조사 · 발표 자료 · 문서 |

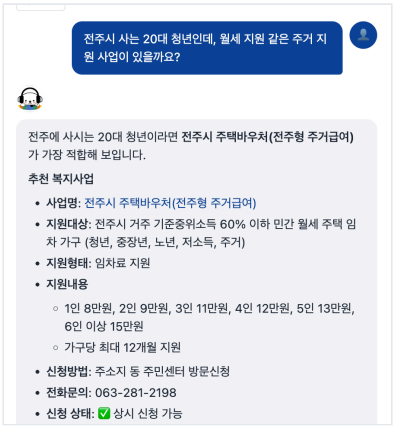
**1,809건**
복지사업 · 매일 03시 갱신

**18,318곳**
복지시설 위치 데이터

**우수상**
캡스톤 우수상 · 기관 표창




질문 하나가 답이 되기까지의 흐름



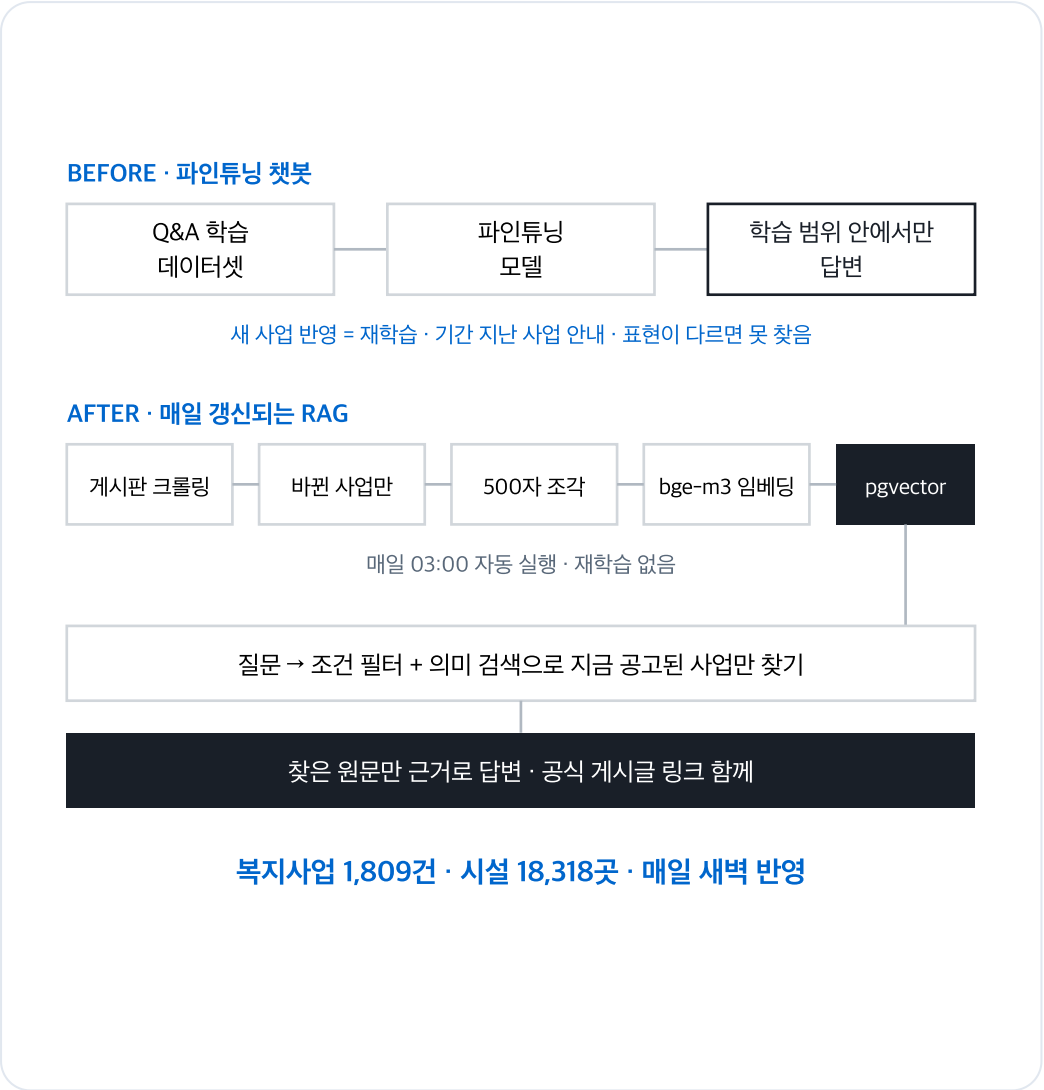




조건을 말하면 받을 수 있는 사업과 신청 방법 안내 / 증상을 말하면 가까운 의료기관을 거리순 안내

# 새 사업을 넣으려면 재학습이 필요했던 파인튜닝 챗봇을 RAG로 전환

문제 해결 3




기존 방식과 새 방식 비교

**문제**

기관의 기존 챗봇은 파인튜닝 기반이라 학습된 Q&A 범위 안에서만 답했습니다. 새 복지사업을 반영하려면 다시 학습해야 해서 신청 기간이 지난 사업을 안내하기도 했습니다. 결국 포털 안에 숨겨진 채 운영되고 있었습니다.

**해결**

**재학습 주기화** 학습 비용이 들고 그 사이 정보는 계속 낡음

**채택** **모델은 그대로 두고 매일 갱신되는 데이터를 찾아 답하는 RAG**

게시판에서 바뀐 사업만 매일 수집하고 500자 이하 조각으로 나눠 bge-m3로 임베딩했습니다. 벡터 DB를 따로 두지 않고 PostgreSQL에 pgvector를 붙여 조건 필터와 의미 검색을 한 쿼리에서 처리했습니다.

전용 벡터 DB의 확장성보다 기관이 관리할 시스템을 하나로 줄이는 쪽을 골랐습니다. 사업과 시설 2만 건 규모와 기관의 운영 인력을 기준으로 한 판단입니다.

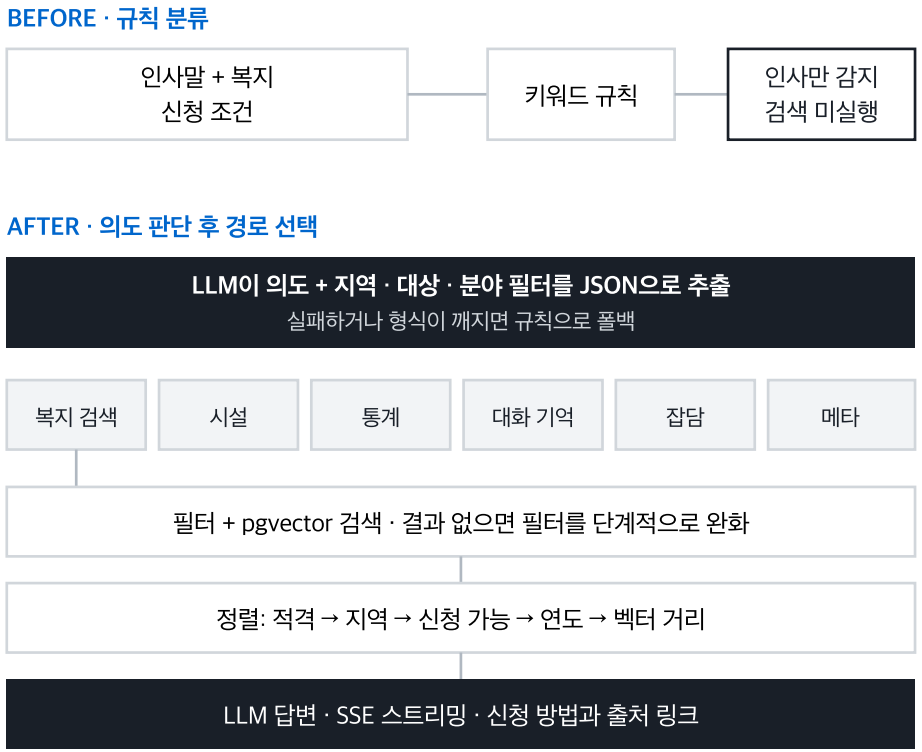
**결과**

재학습 없이 복지사업 1,809건이 매일 새벽 자동으로 반영됩니다. 찾은 원문만 근거로 답하고 공식 게시글 링크를 함께 보여 줍니다. 단순 키워드 검색으로는 0건이던 "치매" 관련 질문에서도 관련 사업을 찾아 안내했습니다. 시군 질문에 도·중앙부처 사업도 함께 찾도록 지역 필터를 고쳐 "남원시 청년 일자리" 검색은 0건에서 13건이 됐습니다.

조건 · 사업 1,809건 · 시설 18,318곳 · 치매 비교는 결과보고서 · 남원 비교는 당시 커밋 기록 기준

# 인사가 섞인 복지 질문도 검색하도록 의도 분류를 개선했습니다

문제 해결 4




질문 처리 흐름 개선 전후

### 문제

인사말과 복지 신청 조건이 함께 들어간 질문이 단순 인사로 분류되어 검색을 건너뛰는 사례를 발견했습니다. 표현별 예외 규칙을 늘리는 방식만으로는 확장성이 부족했습니다.

### 설계 판단

| | |
|---|---|
| 규칙만 추가 | 표현이 달라질 때마다 예외 유지보수 필요 |
| LLM 단독 | 응답 실패·형식 오류에 대한 별도 처리 필요 |
| **선택한 방식** | **LLM으로 의도 추출 + 검증 후 규칙 폴백** |

의도·지역·대상·분야를 JSON으로 추출해 경로를 선택하고, 오류 발생 시 규칙 기반 분류로 이어지도록 했습니다. 사업 검색은 적격 여부와 신청 가능성을 우선해 정렬했습니다.

답변은 **SSE 스트리밍**으로 전송해 생성 과정을 단계적으로 보여줬습니다.

### 결과

질문을 **6개 의도 경로**로 분기해 복지 검색으로 이어지게 했습니다. LLM 오류가 발생해도 규칙 기반 분류로 처리하는 대체 경로를 확보했습니다.

구현 · FastAPI · LangChain · JSON 구조화 · 규칙 폴백 · SSE

# OpenContext · 사내 문서를 밖으로 보내지 않는 AI 검색

개요와 구조

3인 팀 · 오픈소스 개발자대회

2025.08 ~ 2025.12 · github.com/OpenContextAI/open-context

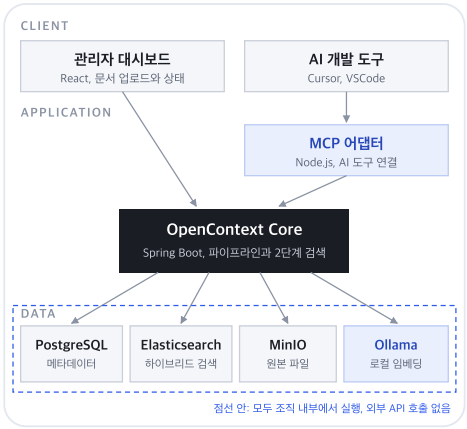
**현장실습 때 외부 MCP 문서 서비스를 써 보니 효과가 컸지만 통제권이 밖에 있었습니다. 그래서 회사가 직접 통제하는 사내 버전을 제안해 팀원들과 만들었습니다.**

| | |
|---|---|
| **Backend** | Java 21 · Spring Boot 3.3 · QueryDSL · Flyway · LangChain4j |
| **검색** | Elasticsearch 8.11 · Ollama bge-m3 로컬 임베딩 · MinIO · PostgreSQL 16 |
| **연동** | MCP (TypeScript SDK) · Cursor · VS Code |
| **Admin** | React 19 · Vite · TanStack Query · Zustand · Tailwind |
| **역할** | **내 담당** 제안 · 하이브리드 검색 엔진 · 2단계 검색 API · DB 모델링과 테스트 · 관리자 화면 · Docker 배포 · 보안 패치<br>**팀원** 문서 수집 파이프라인 · MCP 어댑터 · 공통 에러 처리 |

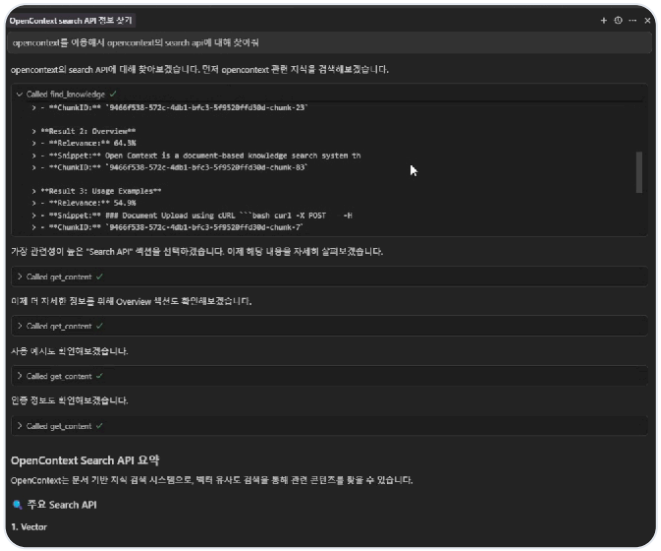
**은상**
학부 작품경진대회

**로컬 처리**
문서 처리 · 임베딩을 조직 안에서

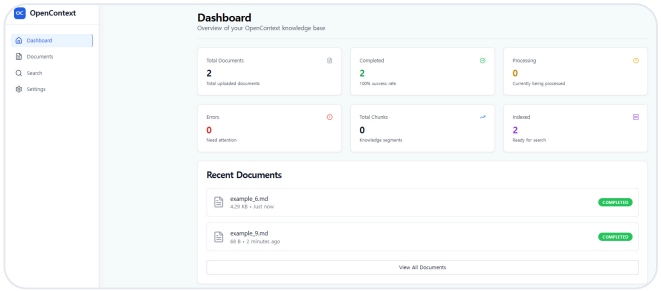
**2단계**
고르고 원문만 가져오기




전체 구조. 점선 안은 모두 조직 내부에서 실행



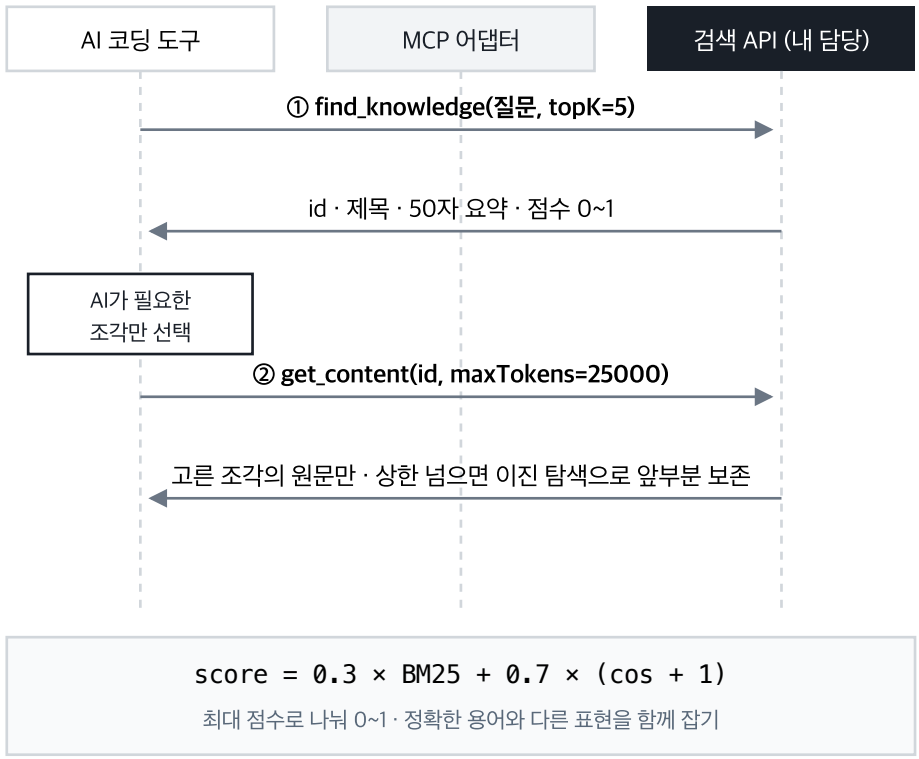

AI 도구가 find_knowledge로 고르고 get_content로 원문만 가져옴




제가 만든 관리자 화면 · 문서 처리 상태 확인

# AI에 필요한 문서만 전달하는 2단계 검색 API를 설계했습니다

문제 해결 5




2단계 검색 호출 순서와 점수 계산

**문제**

사내 문서를 검색한 뒤 모든 원문을 한 번에 전달하면 불필요한 컨텍스트가 커지고, AI가 어떤 자료를 사용할지 선택하기 어려웠습니다.

**해결**

**find_knowledge**에서 제목·요약·관련도 목록을 먼저 반환하고, AI가 선택한 문서만 **get_content**로 원문을 가져가도록 나눴습니다.

검색 API에는 BM25 키워드 점수와 벡터 유사도를 결합했습니다. 결과를 단순 순위만으로 보여주지 않고 관련도 점수도 함께 반환하도록 설계했습니다.

> **핵심 선택** · 검색 결과 탐색과 원문 읽기를 분리해 필요한 컨텍스트만 사용

**결과**

AI 도구가 **관련 자료를 먼저 고르고 원문을 선택적으로 읽는 검색 경로**를 구현했습니다. 문서 처리와 임베딩은 조직 내부 환경에서 동작하도록 구성했고, 팀은 학부 작품경진대회 은상을 받았습니다.

본인 담당 · 하이브리드 검색 API · 데이터 모델링 · 2단계 조회 설계

04

# JH Personal OS · 흩어진 자료를 AI가 정해진 길로만 읽고 쓰게 했습니다

개요와 구조

개인 · 매일 사용 중
2026.09 ~ 현재

**교재 PDF · 지난 AI 대화 · 취업과 행정 서류가 여러 폴더와 서비스에 흩어져 있어 AI가 매번 다른 자료를 보고 답했습니다. 그래서 AI가 필요한 원본만 찾아 읽고, 허용된 명령으로만 기록하는 로컬 커널을 만들었습니다.**

| | |
|---|---|
| 연결한 자료 | 공부 · 취업 · 일정 · 지원정책 · 과거 AI 대화 · 전공 교재 6권 3,321쪽 쪽별 색인 |
| 요청 분류 | 자연어 요청을 도메인 manifest로 매칭. 확실하지 않으면 추측 대신 후보 메뉴 반환 |
| 정보 조회 | 해당 도메인의 원본 · 현재 상태 · 근거만 묶은 context pack을 어떤 AI 도구에든 같은 형식으로 전달 |
| 쓰기 통제 | 도메인마다 허용된 기록 명령만 실행. 이전 버전 확인과 중복 요청 키로 덮어쓰기 차단 |
| 정확성 | 원본 관찰 · 내 발언 · AI 추론을 구분해 저장. AI 추론은 사실로 승격 불가 |
| 검증 · 복구 | 규칙 위반을 잡는 자동 검사 · 병렬 작업 전 Git 스냅샷 · 번들 백업과 복구 검증 |
| Stack | Python · Git · Markdown과 JSON · SQLite · launchd · MCP |

Claude Code · Codex · ChatGPT 중 어느 도구로 물어도 같은 원본과 같은 규칙을 보도록 **도구가 아니라 자료와 권한**을 설계했습니다.




요청이 들어와서 기록되기까지의 흐름

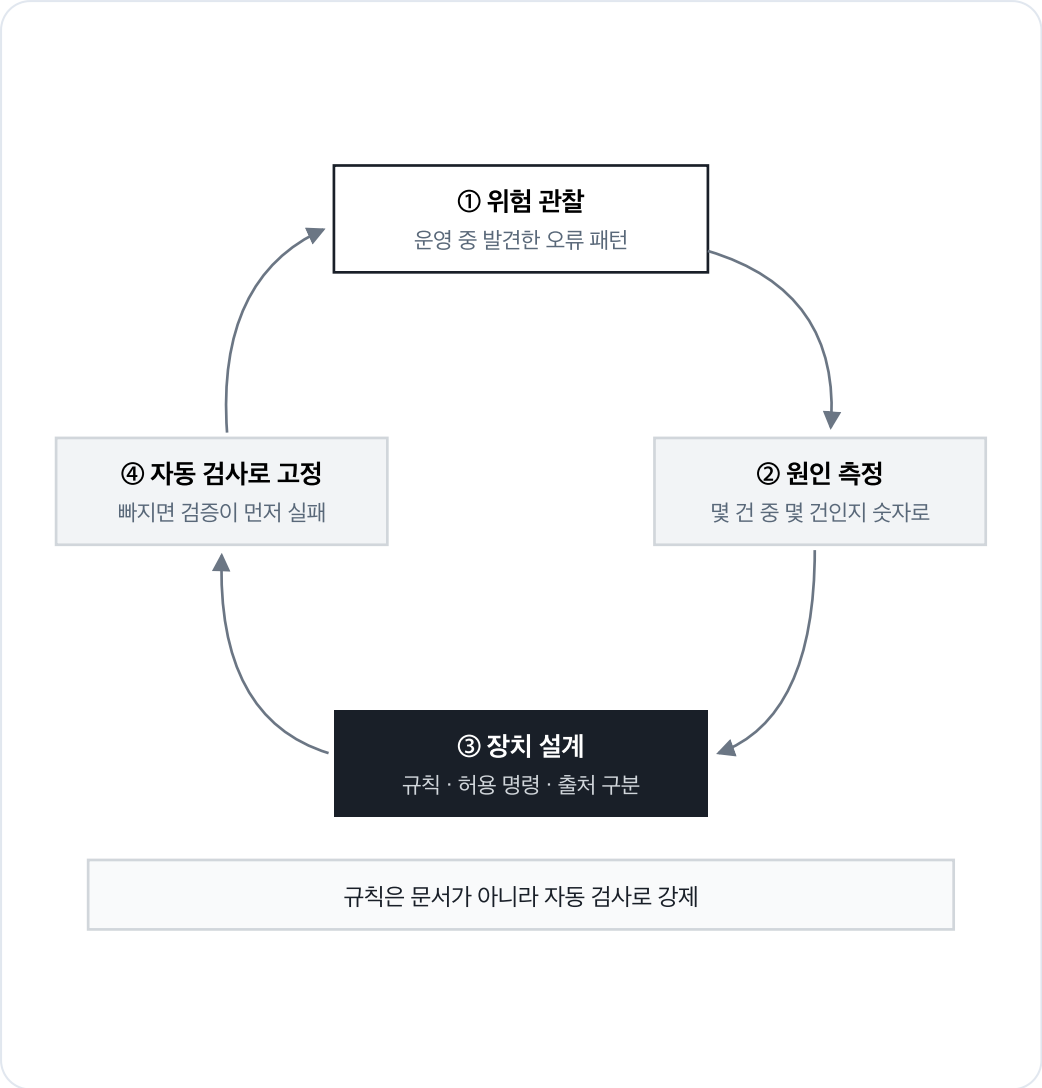
### AI에게 맡기는 일과 내가 쥐는 일

| | |
|---|---|
| AI에게 맡김 | 자료 찾기 · 요약 · 초안 · 코드 구현 · 반복 검사 |
| 내가 쥠 | 무엇을 할지 · 사실인지 · 최종 승인 · 돈과 개인정보가 바뀌는 기록 |
| 나누는 방법 | 계획 → 내 승인 → 실행. 만드는 세션과 검토 세션을 나누고 검토 쪽은 파일을 고치지 않음 |
| 이 환경으로 한 일 | 온라인 스토어 기획 · 개발 · 운영(문제 해결 1·2) · 취업 서류와 이 포트폴리오 작성 · 전공 시험 학습 |

# AI가 틀리기 쉬운 지점마다 검증 장치를 설계했습니다

문제 해결 6






관찰한 위험을 설계와 자동 검사로 바꾸는 순환

### 문제

여러 AI 에이전트가 같은 자료를 읽고 고치면 오래된 요약이 사실처럼 쓰이거나 작업이 서로 덮일 수 있습니다. 규칙을 문서로 주는 것만으로는 AI가 단계를 생략할 수 있어 **코드로 강제하는 장치**가 필요했습니다.

| | |
|---|---|
| **출처 구분** | 원본 관찰 · 내 발언 · AI 추론을 나눠 저장하고, 추론은 사실로 승격할 수 없게 설계 |
| **이미지 판독 관문** | PDF 글자 추출은 같은 쪽 수식을 `At).cos25 7 F*alcos2n58l`로 깨뜨림 → 교재 3,321쪽을 쪽별 이미지로 색인하고 AI가 해당 쪽 이미지를 직접 보고 답하게 함 |
| **의미 기반 라우팅** | 키워드 목록은 실제 요청 22건 중 10건을 놓침 → 도메인 이름 대조와 후보 메뉴로 교체해 막다른 요청 제거 |
| **쓰기 권한 최소화** | 허용된 기록 명령만 실행 · 이전 버전 확인과 중복 요청 키로 덮어쓰기 차단 |
| **병렬 작업 보존** | 여러 AI 세션이 동시에 작업해도 커밋 전 스냅샷으로 미커밋 파일 105개(3.5MB)를 먼저 보존 |
| **한글 경로 정규화** | 파일명 2,382개 중 518개의 유니코드 형식 차이로 생기는 조용한 경로 불일치를 정규화와 상시 검사로 제거 |

### 결과

AI가 틀리기 쉬운 자리마다 **자유도를 줄인 장치**를 두고, 장치가 빠지면 자동 검사가 먼저 실패하게 했습니다. 필요한 조회만 실행하도록 바꿔 응답을 11.1초에서 0.3초로 줄였고, Git 번들 백업과 복구 검증까지 운영합니다.

Python · Git · SQLite · launchd · 2026.09 ~ 현재 매일 사용

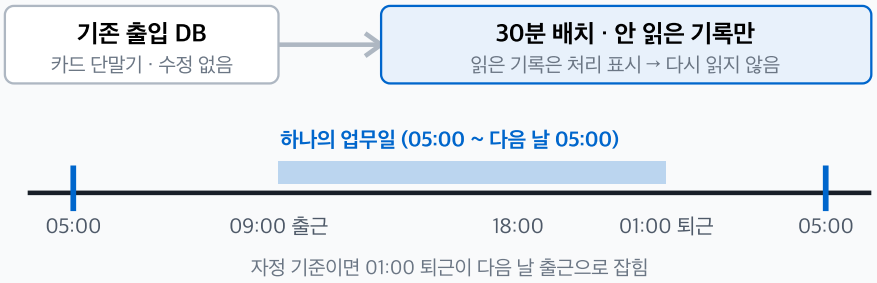
05

# 사내 자리배치도 · 출입 기록만으로 직원 근무 상태를 자동 판정했습니다

문제 해결 7 · 현장실습

(주)텔로스 · 백엔드 전담
2025.07 ~ 2025.08

**업무일 경계와 판정 규칙**




**퇴근 미태깅** 재실로 남은 기록은 지난 업무일만 자동 퇴근 처리

**05:00 이전 실행** 진행 중인 전날은 두고 전전날까지만 마감

**카드 오류** 출근 기록 없이 외출만 찍혔으면 출근으로 간주

출입 기록이 하루 근무 상태가 되기까지

**같은 프로젝트에서 만든 것**

좌석 배정 API(중복 배정 방지) · 직원 · 팀 · 권한 관리 API · JWT 이중 토큰(Access 30분 · Refresh 90일 HttpOnly)과 전체 기기 로그아웃 · 공통 오류 형식과 상황별 오류 코드 · Swagger 문서 · Docker Compose 회사 서버 배포

### 프로젝트

직원 자리 배치와 출근 · 재실 · 퇴근 상태를 한 화면에 보여 주는 **사내 자리배치도 웹**입니다. 백엔드(Spring Boot)를 전담했고 화면(Next.js)은 프론트엔드 선임이 맡았습니다.

### 문제

자리배치도에 직원의 출근·재실·퇴근을 보여 줘야 했지만 회사에 있는 건 출입 카드 기록뿐이었습니다. 운영 중인 출입 시스템은 고칠 수 없었고 기록에는 현실의 예외가 그대로 섞여 있었습니다. 퇴근 카드를 안 찍은 직원은 계속 재실로 남았고 새벽 퇴근은 다음 날 출근처럼 보였습니다.

### 해결

원본 DB는 읽기만 하고 30분마다 아직 처리하지 않은 기록만 가져와 처리 표시를 남겼습니다. 하루를 자정이 아니라 새벽 5시 기준으로 나눠 야근자의 기록이 이틀로 쪼개지지 않게 했습니다. 퇴근 기록이 없으면 지난 업무일만 자동으로 퇴근 처리하고 새벽 5시 전에는 진행 중인 전날을 건드리지 않게 했습니다. 카드 오류로 외출만 찍고 들어온 경우는 출근으로 간주했습니다.

### 결과

원본 시스템을 바꾸지 않고 카드 기록만으로 자리배치도에 직원 상태를 보여 줄 수 있게 됐습니다. 판정 규칙은 실제 근무 방식을 확인하며 다듬었고 회사 서버에 배포했습니다.

조건 · Spring Boot 3.5 · Java 17 · SQL Server · 30분 배치 · 프론트엔드 선임과 2인 개발

**다른 개발 경험**

JeonjuEats · Spring Boot/Vue 기반 주문 서비스, AWS 배포
Calelog · 4인 팀 일정 기반 활동일지 자동화, 백엔드·CI/CD

**배지혁** github.com/qowlgur121